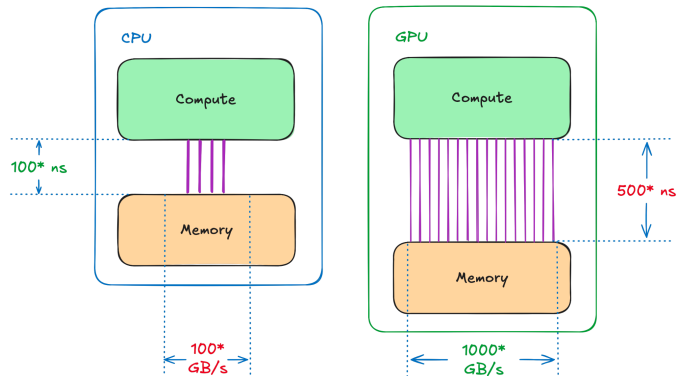


Cuda Notes

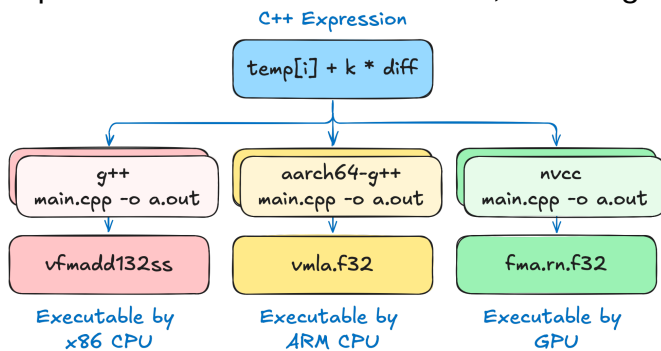
Gombia Matteo

Theory

Comparison between CPU and GPU



Compilation for different architecture, including for GPU



EVERY PROGRAM START IN CPU!!!

1 - thrust

Transform:

CPU

```
std::vector<float> temp{ 42, 24, 50 };

auto op = [=] (float temp) {
    float diff = ambient_temp - temp;
    return temp + k * diff;
};

std::transform(temp.begin(), temp.end(),
               temp.begin(), op);

std::transform(std::execution::par,
               temp.begin(), temp.end(),
               temp.begin(), op);
```

GPU

```
thrust::universal_vector<float> temp{ 42, 24, 50 };

auto op = [=] __host__ __device__ (float temp) {
    float diff = ambient_temp - temp;
    return temp + k * diff;
};

thrust::transform(thrust::device,
                  temp.begin(), temp.end(),
                  temp.begin(), op);
```



Multi thread CPU version:

__host__ -> Compile code for CPU

__device__ -> Compile code for GPU

thrust::host -> Policy: call CPU version

thrust::device -> Policy: call GPU version

Lamda version to use **for_each** on CPU vs GPU:

```
thrust::for_each(thrust::host, vec.begin(), vec.end(), [] __host__ (int val) {
    std::printf("printing %d on %s\n", val, ach::execution_space());
});
```



```
thrust::for_each(thrust::device, vec.begin(), vec.end(), [] __device__ (int val) {
    std::printf("printing %d on %s\n", val, ach::execution_space());
});
```

Median algorithm using **sort**:

```
float median(...)
{
    std::sort(vec.begin(), vec.end());
    return vec[vec.size() / 2];
}
```



```
float median(...)
{
    thrust::sort(thrust::device, vec.begin(), vec.end());
    return vec[vec.size() / 2];
}
```

Median function is called by CPU (-> no __device__ tag)

Iterators to reduce reads/writes compared to pointers (No additional space stored):

Pointers

- Subscript operator leads to memory accesses

```
std::array<int, 3> a{0, 1, 2};
int *ptr = a.data();

std::printf("pointer[0]: %d\n", ptr[0]); // prints 0
std::printf("pointer[1]: %d\n", ptr[1]); // prints 1
```



N

Iterators

```
struct counting_iterator
{
    int operator[](int i)
    {
        return i;
    }
};

counting_iterator it;

std::printf("it[0]: %d\n", it[0]);
std::printf("it[1]: %d\n", it[1]);
```



0

Type of Iterators

Counting Iterator
Mental Model:

```
struct counting_iterator
{
    int operator[](int i)
    {
        return i;
    }
};
```



0

```
counting_iterator it;

std::printf("it[0]: %d\n", it[0]); // prints 0
std::printf("it[1]: %d\n", it[1]); // prints 1
```

Zip iterators
Mental Model:

```
struct zip_iterator
{
    int *a;
    int *b;

    std::tuple<int, int> operator[](int i)
    {
        return {a[i], b[i]};
    }
};
```



2N

```
std::array<int, 3> a{ 0, 1, 2 };
std::array<int, 3> b{ 5, 4, 2 };

zip_iterator it(a.data(), b.data());

std::printf("it[0]: (%d, %d)\n", std::get<0>(it[0]), std::get<1>(it[0]));
```

Transform iterator
Mental Model:

```
struct transform_iterator
{
    int *a;

    int operator[](int i)
    {
        return a[i] * 2;
    }
};
```



N

```
std::array<int, 3> a{ 0, 1, 2 };

transform_iterator it(a.data());

std::printf("it[0]: %d\n", it[0]);
std::printf("it[1]: %d\n", it[1]);
```

Thrust Fancy iterators

`thrust::make_counting_iterator(1)`

(Param: starting value)

```
auto begin = thrust::make_counting_iterator(1);
auto end = begin + 42;

thrust::for_each(thrust::device, begin, end, print)
// prints: 1 2 3 ... 42
```

`thrust::make_zip_iterator(a.begin(), b.begin())`

```
thrust::universal_vector<int> a = {0, 1, 2};
thrust::universal_vector<int> b = {5, 4, 2};

auto begin = thrust::make_zip_iterator(a.begin(), b.begin());
auto end = begin + 42;

thrust::for_each(thrust::device, begin, end, print);
// prints: (0, 5) (1, 4) (2, 2)
```

`thrust::make_transform_iterator(`

```
a.begin(),
[] __host__ __device__(int a) {
    return a * 2;
})

thrust::universal_vector<int> vec = {1, 2, 3, ...};
auto begin = thrust::make_transform_iterator(
    vec.begin(),
    [] __host__ __device__(int value) {
        return value * 2;
    });

auto end = begin + 42;

thrust::for_each(thrust::device, begin, end, print)
// prints: 2 4 6 8 ... 84
```

Combine iterators to compute max differences

```
thrust::universal_vector<float>
unnecessarily_materialized_diff(a.size());

// compute abs differences
thrust::transform(
    thrust::device,
    a.begin(), a.end(), // first input sequence
    b.begin(),          // second input sequence
    unnecessarily_materialized_diff.begin(),
    [][__host__ __device__](float x, float y) {
        return abs(x - y);
    });

// compute max difference
return thrust::reduce(
    thrust::device,
    unnecessarily_materialized_diff.begin(),
    unnecessarily_materialized_diff.end(),
    0.0f, thrust::maximum<float>{});
```



3N



1N



```
auto zip_it = thrust::make_zip_iterator(a.begin(), b.begin());
auto transform_it =
    thrust::make_transform_iterator(
        zip_it,
        [][__host__ __device__](thrust::tuple<float, float> t) {
            return abs(thrust::get<0>(t) - thrust::get<1>(t));
        });

// compute max difference [SAME]
return thrust::reduce(
    thrust::device,
    transform_it.begin(),
    transform_it.end(),
    0.0f, thrust::maximum<float>{});
```



2N



1

Other example: Variance

```
float variance(const thrust::universal_vector<float> &x, float mean)
{
    auto squared_differences = thrust::make_transform_iterator(x.begin(),
        [mean] __host__ __device__ (float value) {
            return (value - mean) * (value - mean);
        });

    return thrust::reduce(thrust::device, squared_differences, squared_differences + x.size() / x.size());
}
```



From vector to pointer

CPU

```
std::vector<int> vec(42);
```

```
int *data = vec.data();
```

—> vec.data: pointer

GPU

```
thrust::universal_vector<int> vec(42);
```

```
thrust::cuda::universal_pointer<int> data = vec.data();
```

```
int *ptr = thrust::raw_pointer_cast(vec.data());
```

—> vet.data(): typed iterator

Tabulate (oven simulation)

Tabulate is equivalent to transform of counting iterator

Transform version:

```
void simulate(int height, int width,
             const thrust::universal_vector<float> &in,
             thrust::universal_vector<float> &out)
{
    const float *in_ptr = thrust::raw_pointer_cast(in.data());

    auto cell_indices = thrust::make_counting_iterator(0);

    thrust::transform(
        thrust::device, cell_indices, cell_indices + in.size(), out.begin(),
        [in_ptr, height, width] __host__ __device__(int id) {
            int column = id % width;          // Convert single cell into 2D coord
            int row = id / width;
            if (row > 0 && column > 0 && row < height - 1 && column < width - 1) {
                float d2tdx2 = in_ptr[row * width + column - 1]
                    - in_ptr[row * width + column] * 2
                    + in_ptr[row * width + column + 1];
                float d2tdy2 = in_ptr[(row - 1) * width + column]
                    - in_ptr[row * width + column] * 2
                    + in_ptr[(row + 1) * width + column];
                return in_ptr[row * width + column] + 0.2f * (d2tdx2 + d2tdy2);
            } else {
                return in_ptr[row * width + column];
            }
        });
}
```

No cell indices -> inside tabulate

```
void simulate(int height, int width,
             const thrust::universal_vector<float> &in,
             thrust::universal_vector<float> &out)
{
    const float *in_ptr = thrust::raw_pointer_cast(in.data());
    thrust::tabulate(
        thrust::device, out.begin(), out.end(),
        [in_ptr, height, width] __host__ __device__(int id) {
            int column = id % width;
            int row = id / width;
            if (row > 0 && column > 0 && row < height - 1 && column < width - 1) {
                float d2tdx2 = in_ptr[row * width + column - 1]
                    - in_ptr[row * width + column] * 2
                    + in_ptr[row * width + column + 1];
                float d2tdy2 = in_ptr[(row - 1) * width + column]
                    - in_ptr[row * width + column] * 2
                    + in_ptr[(row + 1) * width + column];
                return in_ptr[row * width + column] + 0.2f * (d2tdx2 + d2tdy2);
            } else {
                return in_ptr[row * width + column];
            }
        });
}
```

libcu++

Pair for code reuse:

```
__host__ __device__
std::pair<int, int> row_col(int id, int width)
{
    return std::make_pair(id / width, id % width);
}

thrust::tabulate(
    thrust::device, out.begin(), out.end(),
    [in_ptr, height, width] __host__ __device__(int id) {
        auto [row, column] = row_col(id, width);
        ...
    })
```

ERROR: std::make_pair is a host function

```
__host__ __device__
cuda::std::pair<int, int> row_col(int id, int width)
{
    return cuda::std::make_pair(id / width, id % width);
}

thrust::tabulate(
    thrust::device, out.begin(), out.end(),
    [in_ptr, height, width] __host__ __device__(int id) {
        auto [row, column] = row_col(id, width);
        ...
    })
```

cuda::std::make_pair works with both CPU and GPU

mdspan

Multidimensional view

```
cuda::std::array<int, 6> sd{0, 1, 2, 3, 4, 5};
cuda::std::mdspan md(sd.data(), 2, 3); -> It accepts raw pointers
```

```
std::printf("md(0, 0) = %d\n", md(0, 0)); // 0
std::printf("md(1, 2) = %d\n", md(1, 2)); // 5
```

```
std::printf("size = %zu\n", md.size()); // 6
```

```
std::printf("height = %zu\n", md.extent(0)); // 2
std::printf("width = %zu\n", md.extent(1)); // 3
```

Example: previous exercise with mdspan

```

cuda::std::mdspan temp_in(thrust::raw_pointer_cast(in.data()), height, width);
thrust::tabulate(thrust::device, out.begin(), out.end(), [temp_in] __host__ __device__(int id) {
    int column = id % temp_in.extent(1);
    int row = id / temp_in.extent(1);
    if (row > 0 && column > 0 && row < temp_in.extent(0) - 1 && column < temp_in.extent(1) - 1) {
        float d2tdx2 = temp_in(row, column - 1) - 2 * temp_in(row, column) + temp_in(row, column + 1);
        float d2tdy2 = temp_in(row - 1, column) - 2 * temp_in(row, column) + temp_in(row + 1, column);
        return temp_in(row, column) + 0.2f * (d2tdx2 + d2tdy2);
    }
    else {
        return temp_in(row, column);
    }
});
}

```

Best practice: use alias

```

using temperature_grid_f = cuda::std::mdspan<float, cuda::std::dextents<int, 2>>;
using temperature_grid_d = cuda::std::mdspan<double, cuda::std::dextents<int, 2>>;
void simulate(int height, int width, thrust::universal_vector<float> &in, thrust::universal_vector<float> &out){
    temperature_grid_f temp(thrust::raw_pointer_cast(in.data()), height, width);
    ...
}

```

Serial vs Parallel

reduce_by_key for row summation

```

thrust::universal_vector<float> row_ids(height * width);
thrust::tabulate(
    row_ids.begin(), row_ids.end(),
    [=]__host__ __device__(int i) { return i / width; });
    --> Create the key for each row

thrust::universal_vector<float> sums(height);
thrust::reduce_by_key(
    thrust::device,
    row_ids.begin(), row_ids.end(), // input keys
    temp.begin(), // input values
    thrust::make_discard_iterator(), // output keys --> save bandwidth: no writing
    sums.begin()); // output values

```


2N

|


2N


0

We can avoid to store ids and make them as a transform iterator

```

auto row_ids_begin = thrust::make_transform_iterator(
    thrust::make_counting_iterator(0),
    [=]__host__ __device__(int i) {
        return i / width;
    });
auto row_ids_end = row_ids_begin + temp.size();
thrust::universal_vector<float> sums(height);
thrust::reduce_by_key(thrust::device,
    row_ids_begin, row_ids_end,
    temp.begin(), thrust::make_discard_iterator(),
    sums.begin());

```


N

|


N

Transform Output Iterator

```
thrust::make_transform_output_iterator(  
    a.begin(),  
    []__host__ __device__(int a) {  
        return a / 2;  
    })
```

Example: segmented mean

```
struct mean_functor {  
    __host__ __device__ float operator()(float x) const {  
        return x / width;  
    }  
};  
  
thrust::universal_vector<float> means(height);  
auto means_output = thrust::make_transform_output_iterator(means.begin(), mean_functor{});  
  
thrust::reduce_by_key(thrust::device, row_ids_begin, row_ids_end,  
    temp.begin(), thrust::make_discard_iterator(),  
    means_output);
```



MEMORY

`thrust::universal_vector` relies on unified memory -> implicit memory transfer between CPU memory and GPU memory

`thrust::device_vector` -> GPU

`thrust::host_vector` -> CPU

```
// simulate takes longer after the store  
thrust::universal_vector<float> prev = ach::init(height, width);  
thrust::universal_vector<float> next(height * width);  
for (int write_step = 0; write_step < 3; write_step++)  
{  
    ach::store(write_step, height, width, prev);  
    for (int compute_step = 0; compute_step < 3; compute_step++) {  
        ach::simulate(height, width, prev, next);  
        prev.swap(next);  
    }  
}
```



```
thrust::device_vector<float> d_prev = ach::init(height, width);  
thrust::device_vector<float> d_next(height * width);  
thrust::host_vector<float> h_prev(height * width);  
for (int write_step = 0; write_step < 3; write_step++)  
{  
    thrust::copy(d_prev.begin(), d_prev.end(), h_prev.begin());  
    ach::store(write_step, height, width, h_prev);  
    for (int compute_step = 0; compute_step < 3; compute_step++)  
    {  
        ach::simulate(height, width, d_prev, d_next);  
        d_prev.swap(d_next);  
    }  
}
```

2 - Asynchrony and streams

Thrust launches work on the GPU & waits for it to finish (synchronous)

CUB is asynchronous (control flow doesn't wait for the algorithm to finish)

```
thrust::tabulate(thrust::device, out.begin(), out.end(), compute);
```



```
auto cell_ids = thrust::make_counting_iterator(0);
cub::DeviceTransform::Transform(cell_ids, out.begin(), num_cells, compute);
...
cudaDeviceSynchronize(); -> Explicit device synch
```

NSIGHT: provides system-wide view of GPU and CPU activities

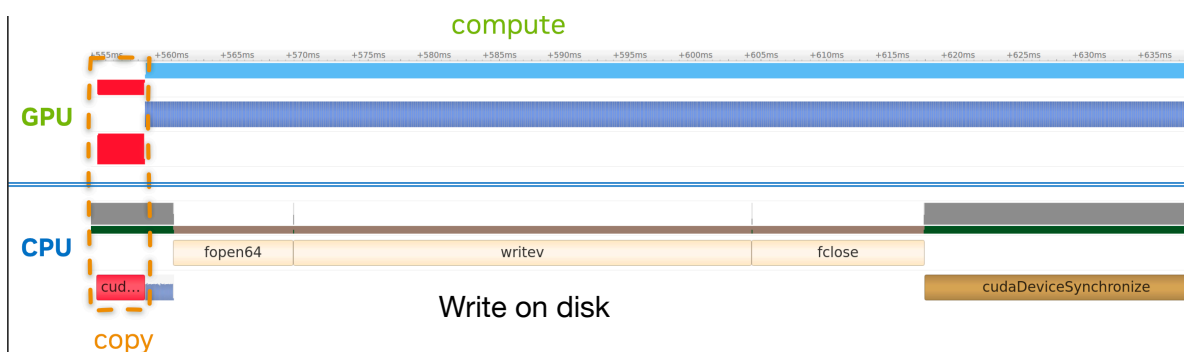
```
!nvcc --extended-lambda -o /tmp/a.out Solutions/compute-
io-overlap.cpp -x cu -arch=native # build executable
!sudo nsys profile --cuda-event-trace=false --force-
overwrite true -o compute-io-overlap /tmp/a.out # run and
profile executable
```

Open compute-io-overlap.nsys-rep with NSIGHT Tool UI

Click and drag to select time regions -> "Filter and Zoom in"

Unfold "CUDA HW" to see details on what GPU is doing

NSIGHT with compute while cpu is writing on disk, then cpu waits:



NVIDIA Tools Extension (**NVTX**)

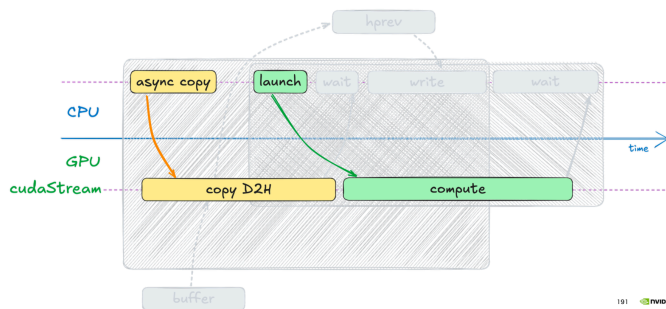
Creates custom ranges in NSIGHT -> annotate copy, compute, write, and wait steps
int main()

```
{
    for (int write_step = 0; write_step < write_steps; write_step++)
    {
        nvtx3::scoped_range r{std::string("write step ") + std::to_string(write_step)};
        ...
    }
}
```

CudaMemcpyAsync: like thrust::copy but async

```
cudaError_t cudaMemcpyAsync(
    void*          dst,  → Destination memory address
    const void*    src,  → Source memory address
    size_t         count, → Size in bytes to copy
    cudaMemcpyKind kind → cudaMemcpyHostToHost
                        → cudaMemcpyHostToDevice
                        → cudaMemcpyDeviceToHost
                        → cudaMemcpyDeviceToDevice -> D2D almost no overhead
);
```

CudaStreams



The compute is ordered on the default stream after the copy even if they don't depend on each other.

```
cudaStream_t copy_stream, compute_stream;
```

I Construction

```
cudaStreamCreate(&compute_stream);
cudaStreamCreate(&copy_stream);
```

II Destruction

```
cudaStreamDestroy(compute_stream);
cudaStreamDestroy(copy_stream);
```

III Synchronisation

```
cudaStreamSynchronize(compute_stream);
cudaStreamSynchronize(copy_stream);
```

Example

```
cudaStream_t copy_stream, compute_stream;
cudaStreamCreate(&compute_stream);
cudaStreamCreate(&copy_stream);

thrust::copy(d_prev.begin(), d_prev.end(), d_buffer.begin()); //(avoid a data race, solve by copying
                                                                dprev -> buffer -> hprev):

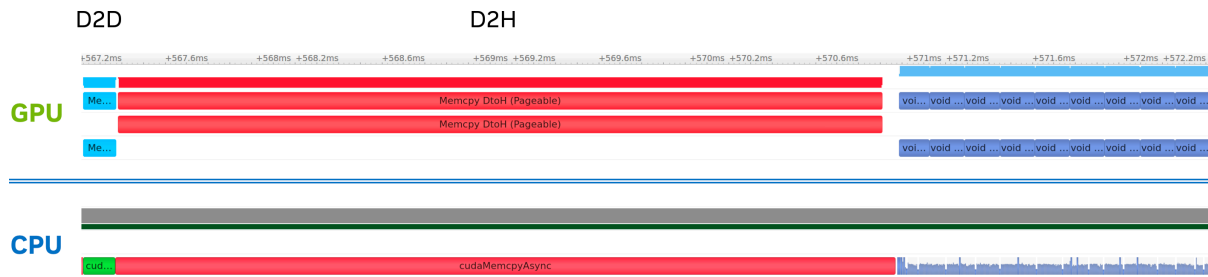
cudaMemcpyAsync(thrust::raw_pointer_cast(htemp.data()),
                thrust::raw_pointer_cast(dbuffer.data()),
                num_cells * sizeof(float), cudaMemcpyDeviceToHost,
                copy_stream);

for (int step = 0; step < steps; step++)
{
    simulate(width, height, dprev, dnext, compute_stream);
    dprev.swap(dnext);
}

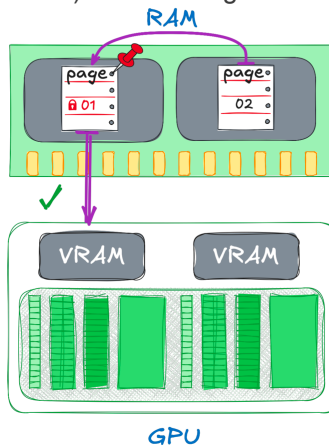
cudaStreamSynchronize(copy_stream);
store(write_step, height, width, hprev);
cudaStreamSynchronize(compute_stream);
```

Pinnable Memory:

Transformation waits the copy to finish even if it's on another stream.



The read from RAM pass through a buffer in pinned memory (to be sure that the page is not on the disk) before being read -> it slows down the copy and obstructs the asynchrony



Solution: mark the memory as pinned memory
Use `thrust::universal_host_pinned_vector`

Pinned memory:

- to make copy to and from device asynchronous
- to make copy to and from device faster

Example (previous example):

```
cudaStream_t copy_stream, compute_stream;
cudaStreamCreate(&compute_stream);
cudaStreamCreate(&copy_stream);
```

```
thrust::universal_host_pinned_vector<float> h_prev(height * width);
```

```
cudaMemcpyAsync(thrust::raw_pointer_cast(d_buffer.data()),
                thrust::raw_pointer_cast(d_prev.data()),
                num_cells * sizeof(float), cudaMemcpyDeviceToDevice);
```

...

3 - CUDA Kernels

`__host__` -> invoked by cpu and executed on cpu

`__device__` -> invoked by gpu and executed on gpu

`__global__` -> invoked by **CPU** and executed on **GPU** (async and parallel)

Example with limited threads:

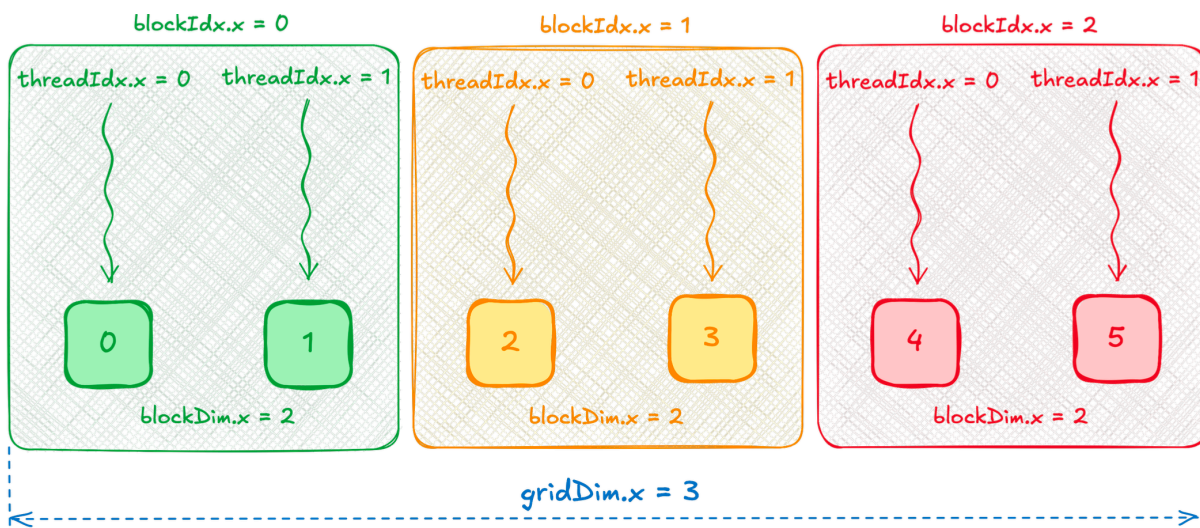
```
const int number_of_threads = 2;
__global__
void block_kernel(ach::temperature_grid_f in, float *out)
{
    int thread_index = threadIdx.x;
    for (int id = thread_index; id < in.size(); id += number_of_threads)
    {
        out[id] = ach::compute(id, in);    -> half to first thread and half on the second one
    }
}

void simulate(ach::temperature_grid_f in, float *out, cudaStream_t stream)
{
    block_kernel<<<1, number_of_threads, 0, stream>>>(in, out);
}
```

`<<< n_blocks, n_threads_per_block, ?, stream>>>`

- **MAX 1024 threads per block**
- Use threads per blocks multiple of 32

`__global__`



```
void grid_kernel(...)
{
    int thread_index = blockDim.x * blockIdx.x + threadIdx.x;
    ...
}
```


To choose the amount of blocks (grid_size) use:

```
void simulate(ach::temperature_grid_f in, float *out, cudaStream_t stream)
{
    int block_size = 256;
    int grid_size = cuda::ceil_div(in.size(), block_size);
    grid_kernel<<<grid_size, block_size, 0, stream>>>(in, out);
}

int ceil_div(int a, int b)
{
    return (a + b - 1) / b;
}
```

Example: Symmetry check on a single row

```
__global__ void symmetry_check_kernel(ach::temperature_grid_f temp, int row)
{
    int column = blockIdx.x * blockDim.x + threadIdx.x;
    if (column < temp.extent(1)) //control to not access memory not allocated
    {
        if (abs(temp(row, column) - temp(temp.extent(0) - 1 - row, column)) > 0.1)
        {
            printf("Error: asymmetry in %d / %d\n", column, temp.extent(1));
        }
    }
}

void symmetry_check(ach::temperature_grid_f temp, cudaStream_t stream)
{
    int width = temp.extent(1);
    int block_size = 256;
    int grid_size = cuda::ceil_div(width, block_size); // width (not temp.size()) -> single row
    int target_row = 0;
    symmetry_check_kernel<<<grid_size, block_size, 0, stream>>>(temp, target_row);
}
```

Detect out of bounds

o Compute sanitiser:

```
nvcc --extended-lambda -g -G -o /tmp/a.out ./src/row_symmetry_check.cu
compute-sanitizer /tmp/a.out
```

o Assertions:

```
#define CCCL_ENABLE_ASSERTIONS
```

—> Runtime checks affect performance (Disabled by default)

SPAN: like a 1D mdspan

```
cuda::std::array<int, 4> a{1, 2, 3, 4};
cuda::std::span<int> span(a.data(), 3); —> (pointer, size)
printf(..., span[0], span.size());
```

```
__global__ void histogram_kernel(
    cuda::std::span<float> temperatures,
    cuda::std::span<int> histogram)
{ ... }
```

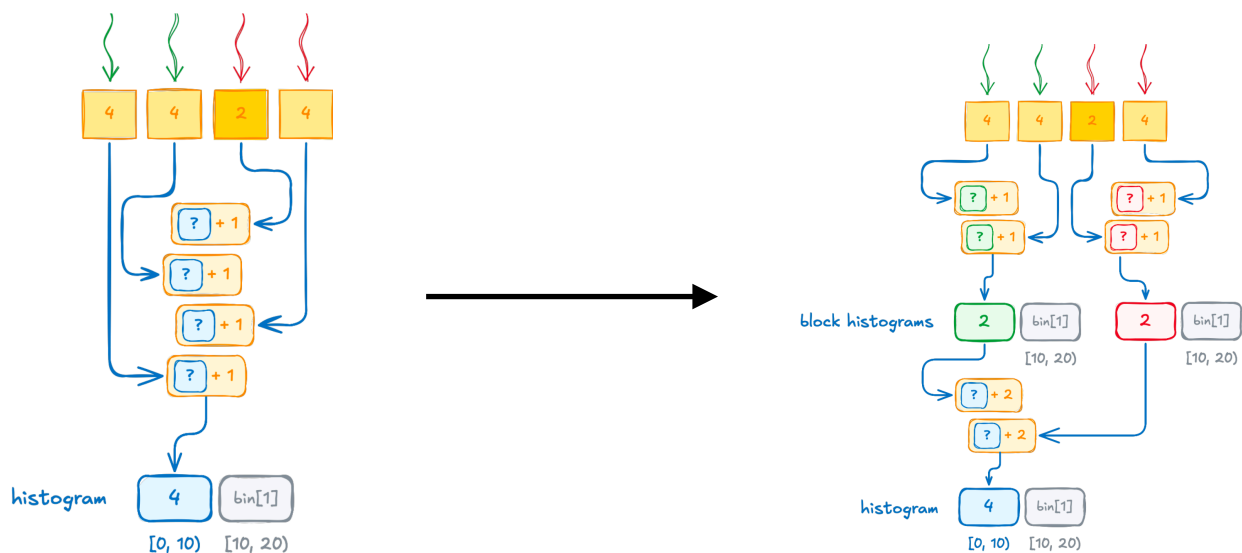
- Includes size information
- Safer alternative to raw pointers
- Reducing common pointer errors
- Lightweight view over a sequence
- Does not own the data, just references it

Avoiding data race: **atomics** makes 1. read 2. modify 3. write steps indivisible

```
cuda::std::atomic_ref<int> ref(histogram[0]);
ref.fetch_add(3);
ref.fetch_sub(2);
ref.fetch_and(1);
```

<pre>__global__ void histogram_kernel(cuda::std::span<float> temperatures, cuda::std::span<int> histogram) { int cell = blockIdx.x * blockDim.x + threadIdx.x; int bin = static_cast<int>(temperatures[cell] / bin_width); int old_count = histogram[bin]; int new_count = old_count + 1; histogram[bin] = new_count; }</pre>	→	<pre>__global__ void histogram_kernel(cuda::std::span<float> temperatures, cuda::std::span<int> histogram) { int cell = blockIdx.x * blockDim.x + threadIdx.x; int bin = static_cast<int>(temperatures[cell] / bin_width); cuda::std::atomic_ref<int> ref(histogram[bin]); ref.fetch_add(1); }</pre>
--	---	---

Problem: Performance (~6 GB/s). → Serialisation of atomics (256 [block size] * 16384 [grid_size] = 4M)
 → Solution: divide bins for each thread block



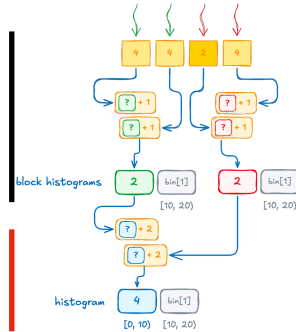
```

__global__ void histogram_kernel(
    cuda::std::span<float> temperatures,
    cuda::std::span<int> block_histograms,
    cuda::std::span<int> histogram)
{
    cuda::std::span<int> block_histogram = block_histograms.subspan(
        blockIdx.x * histogram.size(), histogram.size());

    int cell = blockIdx.x * blockDim.x + threadIdx.x;
    int bin = static_cast<int>(temperatures[cell] / bin_width);
    cuda::std::atomic_ref<int> block_ref(block_histogram[bin]);
    block_ref.fetch_add(1);

    if (threadIdx.x < histogram.size())
    {
        cuda::std::atomic_ref<int> ref(histogram[threadIdx.x]);
        ref.fetch_add(block_histogram[threadIdx.x]);
    }
}

```



—> 1° thread sums block_histogram[0] in the histogram[0]
 2° thread sums block_histogram[1] in the histogram[1]
 ...

Problem: We need to be sure that every thread in the block has finished the updates to enter in the if()

Solution:

__syncthreads()

- > block threads of the block (not in conditional instructions)
- > threads wait the execution of all the others threads in the block

To define the scope of the single atomic ref (and improve performance), we can use:
 Synchronization CUDA Extensions

- | | |
|--------------------------|---------------------|
| • cuda::std::atomic | • cuda::atomic |
| • cuda::std::atomic_ref | • cuda::atomic_ref |
| • cuda::std::atomic_flag | • cuda::atomic_flag |
| • ... | • ... |

CUDA Extension extends cuda::std::atomic_ref with a cuda::thread_scope parameter

```

cuda::atomic_ref<int, thread_scope> ref(...);
    └─> cuda::thread_scope_system (1)
    └─> cuda::thread_scope_device (2)
    └─> cuda::thread_scope_block (3)

```

- (1) Each threads of a given system is related to each other thread on the system (default of cuda::std)
- (2) Each GPU thread is related to each other GPU thread on the same GPU
- (3) Each GPU thread is related to each other GPU thread in the same thread block

Fix with `__syncthreads` and `thread_scope`

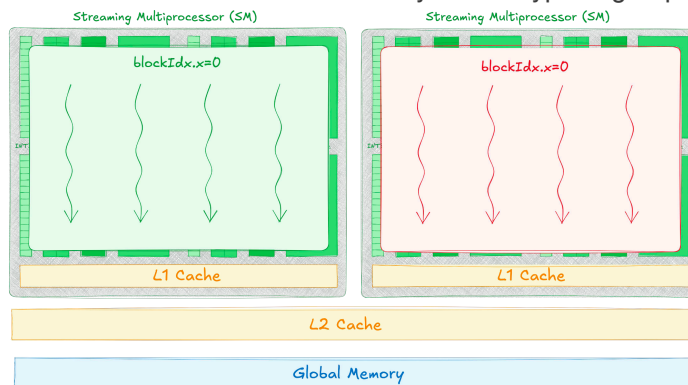
```
cuda::std::span<int> block_histogram = block_histograms.subspan(blockIdx.x * histogram.size(),
                                                                histogram.size());

int cell = blockIdx.x * blockDim.x + threadIdx.x;
int bin = static_cast<int>(temperatures[cell] / bin_width);
cuda::atomic_ref<int, cuda::thread_scope_block> block_ref(block_histogram[bin]);
block_ref.fetch_add(1);
__syncthreads();
if (threadIdx.x < num_bins)
{
    cuda::atomic_ref<int, cuda::thread_scope_device> ref(histogram[threadIdx.x]);
    ref.fetch_add(block_histogram[threadIdx.x]);
}
```

Shared Memory:

`__syncthreads()` blocks only threads in the same block.

Is there other resources shared by certain type of groups? Yes, shared memory.



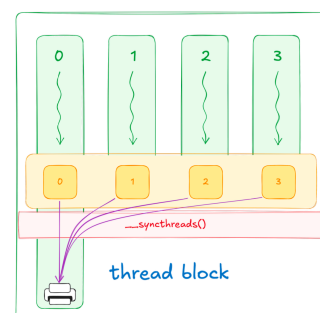
(Cache L1 is shared among threads of the same block)

(VRAM)

In the previous case, "block_histogram" was on Global memory but we need it just in the thread block.
 → Use shared memory / L1 cache

Mock example for shared memory:

```
__global__ void kernel()
{
    __shared__ int shared[4];
    shared[threadIdx.x] = threadIdx.x;
    __syncthreads();
    if (threadIdx.x == 0) {
        for (int i = 0; i < 4; i++) {
            std::printf("shared[%d] = %d\n", i, shared[i]);
        }
    }
}
```



Example: Histogram optimisation (from 100GB/s to 400GB/s)

```
__global__ void histogram_kernel(
    cuda::std::span<float> temperatures,
    cuda::std::span<int> histogram)
{
    __shared__ int block_histogram[num_bins];
    if (threadIdx.x < num_bins) {
        block_histogram[threadIdx.x] = 0; // The first "num_bins" threads initialise the shared mem
    }
    __syncthreads(); // All the other threads need to wait for the initialisation
    int cell = blockIdx.x * blockDim.x + threadIdx.x;
    int bin = static_cast<int>(temperatures[cell] / bin_width);
    cuda::atomic_ref<int, cuda::thread_scope_block>
    block_ref(block_histogram[bin]);
    block_ref.fetch_add(1);
    __syncthreads();
    ...
}
```

COOPERATIVE LIBRARIES

CUB

Cooperative general-purpose
algorithms inside CUDA kernel

cuBLASDx and CUTLASS

Cooperative linear algebra
functions inside CUDA kernel

cuFFTDx

Cooperative fast Fourier
transform inside CUDA kernel

Even in CUDA Kernels, we don't have to implement everything from scratch.

Serial

- One thread invokes algorithm
- One thread executes algorithm

Cooperative (called in CUDA kernels)

- Many threads invoke algorithm
- Many threads execute algorithm

Parallel

- One thread invokes algorithm
- Many threads execute algorithm

TO CONTINUE

TO REWATCH:

- pinned memory
- __syncthread and why using the thread scope